



INSTITUT NATIONAL SUPERIEUR DES
SCIENCES ET TECHNIQUES D'ABECHE

RAPPORT DE PROJET PROFESSIONNELLE ET PERSONNELLE

*3ème Année en Génie Informatique
Option Génie Logiciel*

IA pour la conversation de schéma UML en code fonctionnel

Réalisé par :

Abakar Mahmat Brahim

Encadreur (s) :

Encadrant Professionnel : Dr. MAHAMAT HABIB

Encadrant Pédagogique : M. ABOUBAKAR

Année universitaire : 2025/206

Dédicace

À mes parents,

*pour leur soutien indéfectible, leurs sacrifices et leur amour inconditionnel
tout au long de mon parcours académique.*

À mes frères et sœurs,

pour leur encouragement et leur présence à chaque étape de ma vie.

À mes enseignants de l'INSTA Abéché,

*pour la qualité de la formation dispensée et leur dévouement
au service de la jeunesse tchadienne.*

*À tous ceux qui croient que la technologie
peut transformer le continent africain.*

Je dédie ce travail.

Remerciements

Au terme de ce travail, il m'est agréable d'exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Mes premiers remerciements s'adressent à **Dr. MAHAMAT HABIB**, mon encadrant professionnel, pour sa disponibilité, ses conseils avisés et son suivi tout au long de ce projet. Ses orientations techniques ont été déterminantes dans la réussite de ce travail.

Je remercie également **M. ABOUBAKAR**, mon encadrant pédagogique, pour son soutien académique, ses précieuses remarques et sa rigueur scientifique qui ont grandement contribué à améliorer la qualité de ce rapport.

Je tiens à exprimer ma reconnaissance à l'ensemble du corps enseignant de l'**Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA)**, et plus particulièrement au département de **Génie Logiciel**, pour la qualité de la formation reçue durant ces trois années d'études.

Je remercie sincèrement **l'équipe Groq** pour avoir mis à disposition une API d'inférence IA de haute performance, ainsi que la communauté **open source** de React.js, Flask et Monaco Editor, dont les outils ont été indispensables à la réalisation de ce projet.

Enfin, je remercie chaleureusement **ma famille** pour son soutien moral constant, sa patience et ses encouragements qui ont été une source de motivation inépuisable tout au long de ce travail.

*Abakar Mahamat Brahim
Abéché, Mai 2026*

Résumé

Ce projet présente la conception et le développement d'**UML-to-Code**, une application web full-stack permettant de transformer automatiquement des diagrammes de classes UML en code source fonctionnel, en exploitant l'Intelligence Artificielle générative **Groq Llama 3.3-70b**.

L'application accepte trois formats d'entrée : les fichiers **XMI** exportés depuis StarUML, les **images** PNG/JPG et les **documents PDF**. Elle génère du code **Python** ou **PHP** complet avec classes, attributs, méthodes et héritage, ainsi qu'un **serveur Flask** opérationnel avec routes CRUD automatiques.

L'interface web, développée avec **React.js** et le bundler Vite, intègre le **Monaco Editor** (le moteur de VS Code), un système d'historique local, un mode sombre/clair, une barre de progression en temps réel, des notifications toast et un export en archive ZIP. L'application est déployée en ligne et accessible sur uml-to-code-peach.vercel.app.

Le projet a été réalisé selon le **modèle incrémental** en 5 incréments sur trois semaines, dans le cadre du module de Mini-Projet de Soutenance de la 3^{ème} année en Génie Logiciel à l'INSTA Abéché, Tchad.

Mots-clés : UML, génération de code, Intelligence Artificielle, LLM, Groq, Flask, React.js, MDD, Python, PHP, StarUML, XMI.

Abstract

This project presents the design and development of **UML-to-Code**, a full-stack web application that automatically transforms UML class diagrams into functional source code, leveraging the **Groq Llama 3.3-70b** generative Artificial Intelligence model.

The application accepts three input formats : **XMI** files exported from StarUML, **PNG/JPG images**, and **PDF documents**. It generates complete **Python** or **PHP** code with classes, attributes, methods and inheritance, as well as an operational **Flask server** with automatic CRUD routes.

The web interface, built with **React.js** and Vite, integrates **Monaco Editor** (the VS Code engine), a local history system, dark/light mode, a real-time progress bar, toast notifications, and ZIP archive export. The application is deployed online and accessible at uml-to-code-peach.vercel.app.

The project was carried out using the **incremental model** in 5 increments over three weeks, as part of the Final Project module of the 3rd year Software Engineering program at INSTA Abéché, Chad.

Keywords : UML, code generation, Artificial Intelligence, LLM, Groq, Flask, React.js, MDD, Python, PHP, StarUML, XMI.

Abstract

This project focuses on the development of a web platform entitled **UML-to-Code**, designed to automate the transition between the design phase (UML) and the software development phase. The application allows developers to submit class diagrams in various formats (XMI, PNG/JPG, PDF) and instantly generate structured source code in Python or PHP, as well as a complete Flask server.

The major innovation of this solution lies in the integration of generative Artificial Intelligence (via Groq Llama 3.3-70b), enabling intelligent analysis of diagram images, thus complementing the traditional algorithmic approach based on XMI parsing. The results show a significant time saving in setting up the basic structures of IT projects.

Keywords : UML, Code Generation, Artificial Intelligence, React, Flask, Llama 3.3, Software Engineering.

Table des matières

Liste des figures

Liste des tableaux

Liste des Sigles et Acronymes

API	<i>Application Programming Interface</i> — Interface de programmation
CORS	<i>Cross-Origin Resource Sharing</i> — Partage de ressources entre origines
CRUD	<i>Create, Read, Update, Delete</i> — Opérations de base sur les données
CSS	<i>Cascading Style Sheets</i> — Feuilles de style en cascade
ENV	<i>Environment</i> — Fichier de variables d'environnement
HTML	<i>HyperText Markup Language</i> — Langage de balisage hypertexte
HTTP	<i>HyperText Transfer Protocol</i> — Protocole de transfert hypertexte
IA	Intelligence Artificielle
INSTA	Institut National Supérieur des Sciences et Techniques d'Abéché
JSON	<i>JavaScript Object Notation</i> — Format d'échange de données
LLM	<i>Large Language Model</i> — Grand modèle de langage
MDA	<i>Model-Driven Architecture</i> — Architecture dirigée par les modèles
MDD	<i>Model-Driven Development</i> — Développement dirigé par les modèles
OMG	<i>Object Management Group</i> — Consortium de standardisation
PHP	<i>PHP : Hypertext Preprocessor</i> — Langage de programmation web
PIM	<i>Platform Independent Model</i> — Modèle indépendant de la plateforme
POO	Programmation Orientée Objet
PSM	<i>Platform Specific Model</i> — Modèle spécifique à la plateforme
REST	<i>Representational State Transfer</i> — Style d'architecture pour API web
SPA	<i>Single Page Application</i> — Application web monopage
UML	<i>Unified Modeling Language</i> — Langage de modélisation unifié
URL	<i>Uniform Resource Locator</i> — Adresse web
XMI	<i>XML Metadata Interchange</i> — Format d'échange de modèles UML
XML	<i>eXtensible Markup Language</i> — Langage de balisage extensible
ZIP	<i>Zone Information Protocol</i> — Format d'archive compressée

Introduction Générale

Contexte et problématique

Dans le domaine du Génie Logiciel, la modélisation UML (*Unified Modeling Language*) constitue une étape incontournable du cycle de développement logiciel. Elle permet aux développeurs et architectes de concevoir, documenter et communiquer l'architecture d'un système de manière standardisée, avant même d'écrire la première ligne de code. Cependant, le passage du modèle conceptuel UML au code source représente encore aujourd'hui une tâche manuelle, répétitive et souvent source d'erreurs et d'incohérences.

Cette problématique s'inscrit dans le cadre du **Développement Dirigé par les Modèles** (MDD — *Model-Driven Development*), un paradigme qui préconise la génération automatique de code à partir de modèles de haut niveau. Malgré l'existence de plusieurs outils de transformation UML vers code, ceux-ci restent soit trop complexes à installer et à utiliser, soit limités à un seul langage cible, soit dépourvus de toute intelligence artificielle pour produire un code contextuel et exploitable directement.

Motivation

L'émergence des modèles d'Intelligence Artificielle générative de grande taille (*Large Language Models* — LLM) ouvre de nouvelles perspectives pour automatiser et enrichir ce processus de transformation. En exploitant la capacité de ces modèles à comprendre la structure d'un diagramme UML et à générer du code source cohérent, il devient possible de franchir un cap significatif dans l'automatisation du développement logiciel.

C'est dans ce contexte que s'inscrit le présent projet, réalisé dans le cadre du module de Mini-Projet de Soutenance de la troisième année en Génie Logiciel à l'Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA), Tchad.

Objectifs

Ce projet a pour objectif principal de concevoir et développer une application web full-stack, baptisée **UML-to-Code**, capable de transformer automatiquement un diagramme de classes UML en code source fonctionnel, en exploitant le modèle d'IA générative **Groq Llama 3.3-70b**.

Plus précisément, les objectifs visés sont les suivants :

- Accepter plusieurs formats d'entrée : fichier XML exporté depuis StarUML, image PNG/JPG, ou document PDF

- Générer du code source **Python** ou **PHP** complet avec classes, attributs, méthodes et relations d'héritage
- Générer automatiquement un **serveur Flask** avec les routes CRUD correspondantes
- Proposer une interface web moderne, intuitive et responsive avec un éditeur de code intégré (**Monaco Editor**)
- Permettre l'export du projet complet en archive **ZIP** prête à déployer
- Déployer l'application en ligne et la rendre accessible au grand public

Structure du rapport

Ce rapport est organisé en trois chapitres principaux, encadrés par une introduction et une conclusion générales :

- **Chapitre 1 — Étude préliminaire et analyse des besoins** : présente le contexte général du projet, l'analyse de l'existant, la problématique identifiée, les acteurs du système et les exigences fonctionnelles et non fonctionnelles.
- **Chapitre 2 — Conception et Modélisation** : décrit les choix de conception, le cycle de vie retenu, l'architecture globale du système, ainsi que les diagrammes UML de conception (Use Case, Classes, Séquence).
- **Chapitre 3 — Réalisation et Implémentation** : présente l'environnement de développement, les détails d'implémentation du backend et du frontend, le déploiement de l'application, ainsi que les tests et résultats obtenus.

Chapitre 1

Étude Préliminaire et Analyse des Besoins

Introduction

Dans ce premier chapitre, nous présentons l'Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA), notre établissement de formation. Nous exposons ensuite le contexte général du projet, l'analyse de l'existant, la problématique identifiée, ainsi que les besoins fonctionnels et non fonctionnels du système à concevoir.

1 Contexte Institutionnel : l'INSTA

1.1 Présentation générale

L'Institut National Supérieur des Sciences et Techniques d'Abéché (INSTA) est un établissement public d'enseignement supérieur à caractère scientifique et technique, situé dans la ville d'Abéché (province du Ouaddaï), au Nord-Est du Tchad. Créé initialement sous le nom d'Institut Universitaire des Sciences et Techniques d'Abéché (IUSTA) en 1997, il a été rebaptisé INSTA par l'Ordonnance N°004/PR/MES/2015 du 02 mars 2015.

1.2 Missions de l'INSTA

L'institut poursuit plusieurs missions fondamentales :

- **Formation académique et professionnelle** : former des étudiants dans divers domaines techniques et scientifiques, en vue de répondre aux besoins du marché de l'emploi.
- **Recherche et innovation** : promouvoir la recherche scientifique et valoriser les résultats pour répondre aux besoins du pays.
- **Partenariats** : établir des collaborations avec des institutions nationales et internationales pour renforcer la qualité de l'enseignement.
- **Contribution au développement** : fournir au marché de l'emploi des ingénieurs compétents capables d'apporter des solutions aux problématiques locales.

1.3 Offres de formation

L'INSTA offre deux cycles principaux :

- **Premier cycle** : Licence professionnelle (3 ans) formant des licenciés professionnels dans les domaines scientifiques et techniques.
- **Deuxième cycle** : Master (2 ans), divisé en Master professionnel et recherche, dans les domaines de Génie Électrique, Génie Mécanique, Génie Énergétique, Sciences Biomédicales et Sciences et Techniques d'Élevage.

1.4 Départements de l'INSTA

L'institut est composé de plusieurs départements :

- Département de Génie Informatique (GI) — option Génie Logiciel
- Département de Génie Mécanique (GM)
- Département de Génie Électrique (GE)
- Département de Génie Énergétique (GEN)
- Département de Télécommunication et Réseau (TLC)
- Département Conception, Installation et Maintenance des Systèmes (CIMS)
- Département des Sciences et Techniques d'Élevage (STE)
- Département des Sciences Biomédicales et Pharmaceutiques (SBMP)

2 Contexte Général du Projet

2.1 Le Génie Logiciel et la modélisation UML

Le Génie Logiciel est la discipline qui s'intéresse à la conception, au développement, à la maintenance et à la gestion de systèmes logiciels de qualité. Parmi les pratiques fondamentales de cette discipline, la modélisation occupe une place centrale. Elle permet de représenter un système sous forme abstraite avant son implémentation, facilitant ainsi la communication entre les parties prenantes et la détection précoce des erreurs de conception.

L'UML (*Unified Modeling Language*) est le langage de modélisation le plus utilisé dans l'industrie du logiciel. Il propose un ensemble de diagrammes standardisés permettant de décrire l'architecture statique et dynamique d'un système logiciel. Pour notre projet, nous avons utilisé trois types de diagrammes :

Diagramme	Objectif	Type
Diagramme de classes	Représenter la structure statique du système : classes, attributs, méthodes et relations	Statique
Diagramme de cas d'utilisation	Décrire les interactions entre les acteurs et le système	Fonctionnel
Diagramme de séquence	Illustrer les échanges entre composants dans un ordre temporel	Dynamique

TABLE 1.1 – Diagrammes UML utilisés dans le projet

2.2 Le Développement Dirigé par les Modèles (MDD)

Le **Développement Dirigé par les Modèles** (MDD — *Model-Driven Development*) est un paradigme de développement logiciel qui place le modèle au cœur du processus de développement. L'idée fondatrice du MDD est que le modèle n'est plus un simple document de documentation, mais un artefact de première classe à partir duquel le code source peut être généré automatiquement.

Dans ce paradigme, on distingue :

- Le **PIM** (*Platform Independent Model*) : le modèle UML indépendant de toute plateforme
- Le **PSM** (*Platform Specific Model*) : le code source généré pour une plateforme cible (Python, PHP, etc.)
- La **transformation** : le processus automatique qui convertit le PIM en PSM

Notre projet s'inscrit exactement dans cette approche : le diagramme UML est le PIM, et le code Python ou PHP généré est le PSM.

2.3 L'Intelligence Artificielle générative

L'avènement des **Grands Modèles de Langage** (LLM — *Large Language Models*) représente une révolution dans le domaine de l'Intelligence Artificielle. Ces modèles, entraînés sur des milliards de lignes de texte et de code source, sont capables de comprendre et de générer du code source de manière contextuelle et cohérente.

Dans le cadre de ce projet, nous exploitons le modèle **Groq Llama 3.3-70b**, un LLM open source de 70 milliards de paramètres développé par Meta et hébergé sur l'infrastructure d'inférence ultra-rapide **Groq**, qui permet des temps de génération de l'ordre de 2 à 5 secondes.

2.4 Le format XMI

Le format **XMI** (*XML Metadata Interchange*) est un standard défini par l'OMG (*Object Management Group*) pour l'échange de modèles UML entre différents outils de modélisation. C'est un fichier texte structuré en XML qui contient la description complète d'un modèle UML : classes, attributs, méthodes, relations et héritage.

StarUML, l'outil de modélisation que nous utilisons, permet d'exporter les diagrammes de classes au format XMI 2.1. Notre application parse ce fichier pour en extraire automatiquement la structure orientée objet et générer le code correspondant.

3 Analyse de l'Existant

3.1 Outils existants de transformation UML vers code

Plusieurs outils de transformation UML vers code existent sur le marché. Nous en avons analysé les principaux afin d'identifier leurs limites :

Outil	Langages générés	Format entrée	Limites principales
Acceleo (Eclipse)	Java, Python, PHP	XMI/EMF	Très complexe, lourd à installer
AndroMDA	Java EE uniquement	XMI	Limité à Java, pas d'IA
GenMyModel	Java, Python	En ligne	Payant, pas open source
StarUML Plugins	JavaScript, Java	.mdj	Limité, pas d'IA générative
Solutions manuelles	Tous langages	Tous formats	Lent, source d'erreurs

TABLE 1.2 – Comparatif des outils existants de transformation UML vers code

3.2 Limites des solutions existantes

L'analyse comparative révèle plusieurs lacunes communes :

- **Complexité d'installation** : des outils comme Acceleo nécessitent une installation lourde et une configuration complexe
- **Absence d'IA** : aucun outil n'intègre de modèle d'IA générative pour produire un code contextuel
- **Formats d'entrée limités** : la plupart n'acceptent que le format XMI, excluant les images et les PDF
- **Coût** : certaines solutions performantes sont payantes et propriétaires

- **Pas de génération de serveur** : aucun outil ne génère automatiquement un serveur REST opérationnel

3.3 Valeur ajoutée de notre solution

Face à ces limitations, **UML-to-Code** apporte une réponse innovante :

- Intégration d'une IA générative (**Groq Llama 3.3-70b**) pour un code contextuel et complet
- Support de plusieurs formats d'entrée : XMI, image PNG/JPG, PDF
- Génération automatique d'un serveur Flask complet avec routes CRUD
- Interface web moderne avec Monaco Editor, historique local, mode sombre/clair
- Export en archive ZIP du projet complet prêt à déployer
- Déploiement en ligne accessible via uml-to-code-peach.vercel.app
- Solution légère, open source et sans installation complexe

4 Spécification des Besoins

4.1 Acteurs du système

gray !30 Acteur	Rôle	Interactions principales
Utilisateur	Étudiant ou développeur utilisant l'outil	Upload fichier, choix options, génération, visualisation Monaco Editor, copie, téléchargement, consultation historique
API Groq (IA)	Modèle Llama 3.3-70b	Reçoit la description UML extraite, retourne le code généré
Backend Flask	Serveur Python	Orchestration, parsing XMI/image/PDF, appel API Groq, génération ZIP
localStorage	Stockage navigateur	Persiste l'historique des 10 dernières générations et le thème

TABLE 1.3 – Acteurs du système UML-to-Code

4.2 Exigences Fonctionnelles

Module Upload

ID	Exigence	Priorité
EF-01	Accepter les fichiers XMI exportés depuis StarUML	Haute
EF-02	Accepter les images PNG et JPG de diagrammes UML	Haute
EF-03	Accepter les fichiers PDF contenant un diagramme UML	Moyenne
EF-04	Valider le format du fichier avant traitement	Haute
EF-05	Afficher un message d'erreur en cas de format invalide	Haute
EF-06	Afficher un aperçu visuel du diagramme pour PNG/JPG	Moyenne

TABLE 1.4 – Exigences fonctionnelles — Module Upload

Module Parsing

ID	Exigence	Priorité
EF-07	Parser les classes, attributs et méthodes depuis un XMI	Haute
EF-08	Extraire les relations d'héritage entre classes	Haute
EF-09	Extraire les associations et agrégations	Moyenne
EF-10	Envoyer l'image/PDF directement à Groq (vision IA)	Haute

TABLE 1.5 – Exigences fonctionnelles — Module Parsing

Module Génération IA

gray !30 ID	Exigence	Priorité
EF-11	Appeler l'API Groq (Llama 3.3-70b) avec la description UML	Haute
EF-12	Générer le code Python complet avec __init__, getters, setters	Haute
EF-13	Générer le code PHP avec visibilité, constructeur, getters	Haute
EF-14	Générer un serveur Flask avec les routes CRUD	Haute
EF-15	Permettre le choix du langage cible : Python ou PHP	Haute
EF-16	Permettre le choix du mode : Algorithmique ou IA Groq	Haute

TABLE 1.6 – Exigences fonctionnelles — Module Génération IA

Module Interface Utilisateur

ID	Exigence	Priorité
EF-17	Afficher le code dans Monaco Editor (VS Code intégré)	Haute
EF-18	Afficher onglet Code et onglet Serveur Flask séparément	Haute
EF-19	Permettre la copie du code en un clic	Haute
EF-20	Permettre le téléchargement du fichier .py ou .php	Haute
EF-21	Permettre l'export en archive ZIP	Haute
EF-22	Afficher une barre de progression en temps réel	Haute
EF-23	Implémenter un historique local des 10 dernières générations	Haute
EF-24	Implémenter le mode sombre / mode clair	Moyenne
EF-25	Afficher des notifications toast	Moyenne

TABLE 1.7 – Exigences fonctionnelles — Module Interface

4.3 Exigences Non Fonctionnelles

gray !30 Catégorie	Exigence	Critère mesurable
Performance	Temps de réponse de la génération	< 15 secondes par diagramme
Disponibilité	Application accessible en ligne	24h/24, 7j/7 via Vercel
Fiabilité	Taux de succès de la génération	> 90% sur les cas de test
Maintenabilité	Code source documenté et structuré	Commentaires sur chaque fonction
Portabilité	Fonctionne sur Windows et Linux	Testé sur les deux systèmes
Sécurité	Clé API en variable d'environnement	Fichier .env non versionné
Utilisabilité	Interface intuitive sans formation	Prise en main < 5 minutes
Responsivité	Interface adaptée mobile et desktop	Testé de 320px à 1440px

TABLE 1.8 – Exigences non fonctionnelles

Conclusion

Dans ce premier chapitre, nous avons présenté l'établissement d'accueil INSTA et exposé le contexte général du projet. Nous avons analysé les solutions existantes de transformation UML vers code et identifié leurs limites, ce qui nous a permis de justifier la valeur ajoutée de notre solution UML-to-Code. Enfin, nous avons spécifié les besoins fonctionnels et non fonctionnels du système à concevoir. Le chapitre suivant sera consacré à la conception et la modélisation du système.

Chapitre 2

Conception et Modélisation

Introduction

Dans ce chapitre, nous présentons les choix de conception retenus pour le projet UML-to-Code. Nous décrivons le cycle de vie adopté, l'architecture globale du système, puis nous modélisons le système à travers les diagrammes UML de conception : diagramme de cas d'utilisation, diagramme de classes et diagramme de séquence. Nous terminons par une description détaillée des modules principaux.

1 Choix du Cycle de Vie

1.1 Définition du cycle de vie

Le cycle de vie de développement d'une application représente la période de temps qui recouvre la vie d'une application : sa naissance, sa construction, son exploitation, sa maintenance et son retrait. Le choix du cycle de vie est une décision fondamentale qui détermine la manière dont les activités de développement sont organisées et enchaînées dans le temps.

1.2 Comparaison des cycles de vie

Plusieurs modèles de cycle de vie existent, chacun adapté à des contextes spécifiques. Le tableau ci-dessous présente une comparaison des principaux modèles :

Cycle de vie	Durée idéale	Solo	3 semaines	Décision
Cascade (Waterfall)	6+ mois	Oui	Non	Rejeté
Cycle en V	3+ mois	Oui	Limite	Rejeté
Agile / Scrum	Continu	Limite	Oui	Rejeté
Incrémental	1–6 mois	Oui	Oui	Retenu

TABLE 2.1 – Comparaison des cycles de vie

1.3 Modèle Incrémental — Justification

Le modèle de cycle de vie retenu pour ce projet est le **modèle incrémental**. Ce choix est motivé par trois raisons principales :

- **Durée courte du projet** : trois semaines de développement nécessitent des livraisons rapides et concrètes à chaque étape
- **Projet réalisé en solo** : les cycles nécessitant une équipe (comme Scrum) ne sont pas adaptés à un développeur unique
- **Incertitudes techniques** : l'intégration de l'IA générative (Groq) et des interfaces avancées (Monaco Editor) impose un ajustement progressif

Le modèle incrémental permet de construire le système par couches successives. Chaque incrément produit une version fonctionnelle et testée du logiciel, enrichie de nouvelles fonctionnalités. Contrairement au modèle en cascade, il permet de valider les choix techniques tôt et d'ajuster la trajectoire si nécessaire.

1.4 Les 5 incréments du projet

Incrément	Période	Fonctionnalités ajoutées	Version
0 — Préparation	05–07 mai	Questions prof, étude XMI, recherche, CDC	v0.0
1 — Fondations	08–14 mai	Architecture, parser XMI, modèle de données, test API Groq	v0.1
2 — Cœur IA & UI	15–21 mai	Groq complet, générateurs Python+PHP, Flask auto, React, Monaco Editor, Historique	v0.2
3 — Finalisation	22–26 mai	Tests, export ZIP, thème sombre/clair, animations, rapport, slides	v1.0
4 — Améliorations	27 mai+	Optimisations UI, statistiques, toast, déploiement Vercel	v1.1

TABLE 2.2 – Les 5 incréments du projet UML-to-Code

2 Architecture Globale du Système

2.1 Vue d'ensemble

Le système UML-to-Code suit une **architecture client-serveur en 3 couches** :

- **Couche Présentation** : Interface React.js avec design Glassmorphism, Monaco Editor, historique localStorage, thème sombre/clair et animations
- **Couche Métier** : Serveur Flask Python assurant l'orchestration, le parsing XMI/image/PDF, l'appel à l'API Groq et la génération ZIP
- **Couche IA** : API Groq (Llama 3.3-70b) assurant la génération intelligente du code Python, PHP et Flask

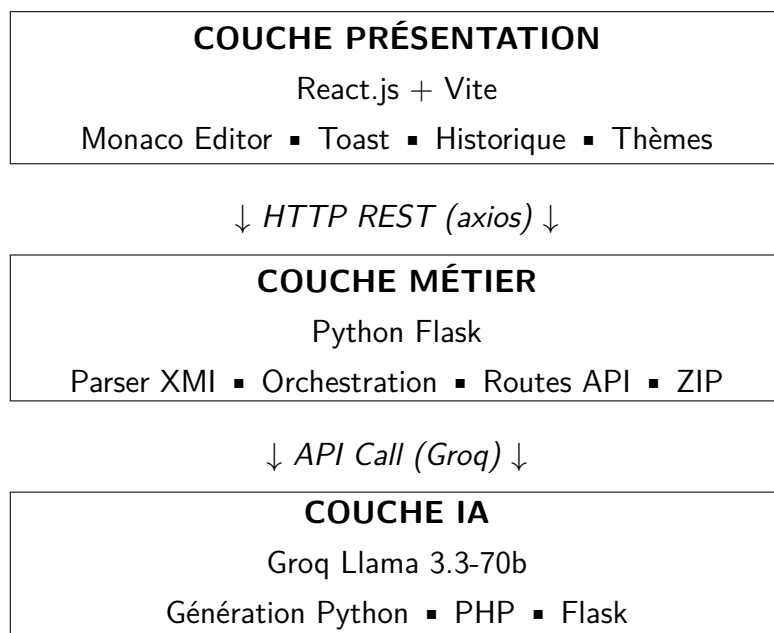


FIGURE 2.1 – Architecture globale en 3 couches du système UML-to-Code

2.2 Stack technologique

Couche	Technologie	Version	Rôle
Frontend	React.js + Vite	6.x	Interface utilisateur web
Éditeur	Monaco Editor	Latest	Éditeur de code intégré (VS Code)
HTTP Client	Axios	Latest	Requêtes Frontend → Backend
Backend	Python Flask	3.0.0	Serveur API REST
IA	Groq Llama 3.3-70b	Latest	Génération de code intelligent
Parsing XML	xml.etree	Natif Python	Extraction classes UML depuis XML
Parsing Image	PyMuPDF	1.27	Conversion PDF → image
Export	zipfile	Natif Python	Export projet ZIP
Stockage	localStorage	Natif Web	Historique local + thème
Déploiement	Vercel + Render	—	Hébergement Frontend + Backend
Versioning	Git + GitHub	—	Gestion du code source

TABLE 2.3 – Stack technologique du projet UML-to-Code

3 Modélisation UML

3.1 Diagramme de Cas d'Utilisation

Définition

Le diagramme de cas d'utilisation décrit les utilisations requises d'un système, ou ce qu'un système est supposé faire. Les principaux concepts de ce diagramme sont les acteurs, les cas d'utilisation et les sujets. Il modélise à **QUOI sert le système**.

Identification des cas d'utilisation

Dans l'application UML-to-Code, nous avons identifié les cas d'utilisation suivants :

gray !30 N°	Acteur	Cas d'utilisation
1	Utilisateur	Charger un fichier UML (XMI, PNG, JPG, PDF)
2	Utilisateur	Choisir le format d'entrée
3	Utilisateur	Choisir le langage cible (Python ou PHP)
4	Utilisateur	Générer le code source
5	Utilisateur	Générer le serveur Flask automatique
6	Utilisateur	Visualiser le code généré (Monaco Editor)
7	Utilisateur	Copier le code généré
8	Utilisateur	Télécharger le fichier généré (.py / .php)
9	Utilisateur	Exporter le projet en ZIP
10	Utilisateur	Consulter l'historique des générations
11	API Groq	Recevoir la description UML et retourner le code

TABLE 2.4 – Cas d'utilisation identifiés du système UML-to-Code

Diagramme de cas d'utilisation

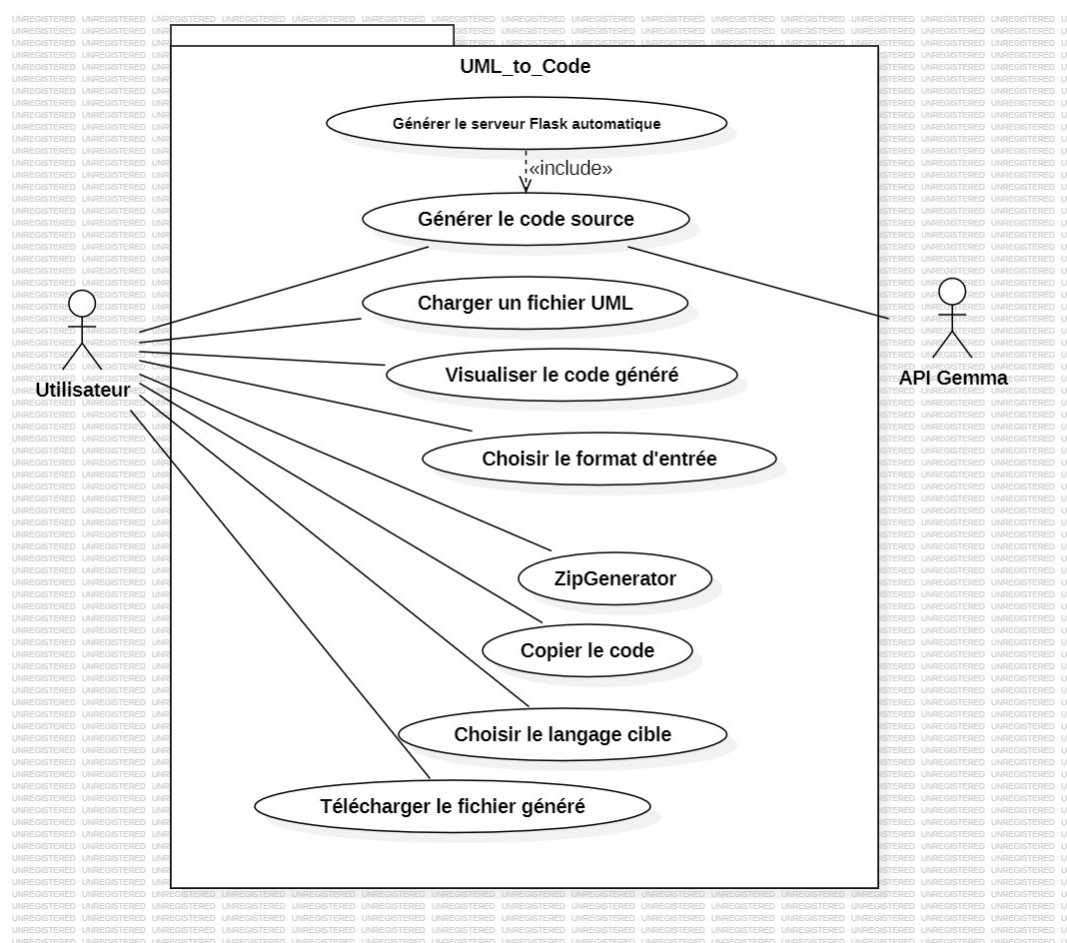


FIGURE 2.2 – Diagramme de cas d'utilisation — UML-to-Code

3.2 Diagramme de Classes

Définition

Le diagramme de classes est le diagramme le plus important pour les méthodes d'analyse orientées objet. Il permet de spécifier **QUI intervient** à l'intérieur du système et représente une vue statique du système d'information. Il modélise les classes, leurs attributs, leurs méthodes et leurs relations.

Description des classes

Les classes principales du système sont :

- **XMIParser** : responsable du parsing des fichiers XMI et de l'extraction des classes UML
- **FileUploadHandler** : gère l'upload, la validation et la détection du format des fichiers
- **UMLClass, UMLAttribute, UMLMethod** : le modèle de données représentant un diagramme UML parsé
- **GroqClient** : client de l'API Groq pour la génération de code par IA

- **BaseGenerator** : classe abstraite dont héritent *PythonGenerator* et *PHPGenerator*
- **FlaskServerGenerator** : génère automatiquement le serveur Flask avec routes CRUD
- **ZipGenerator** : regroupe le code et le serveur Flask dans une archive ZIP

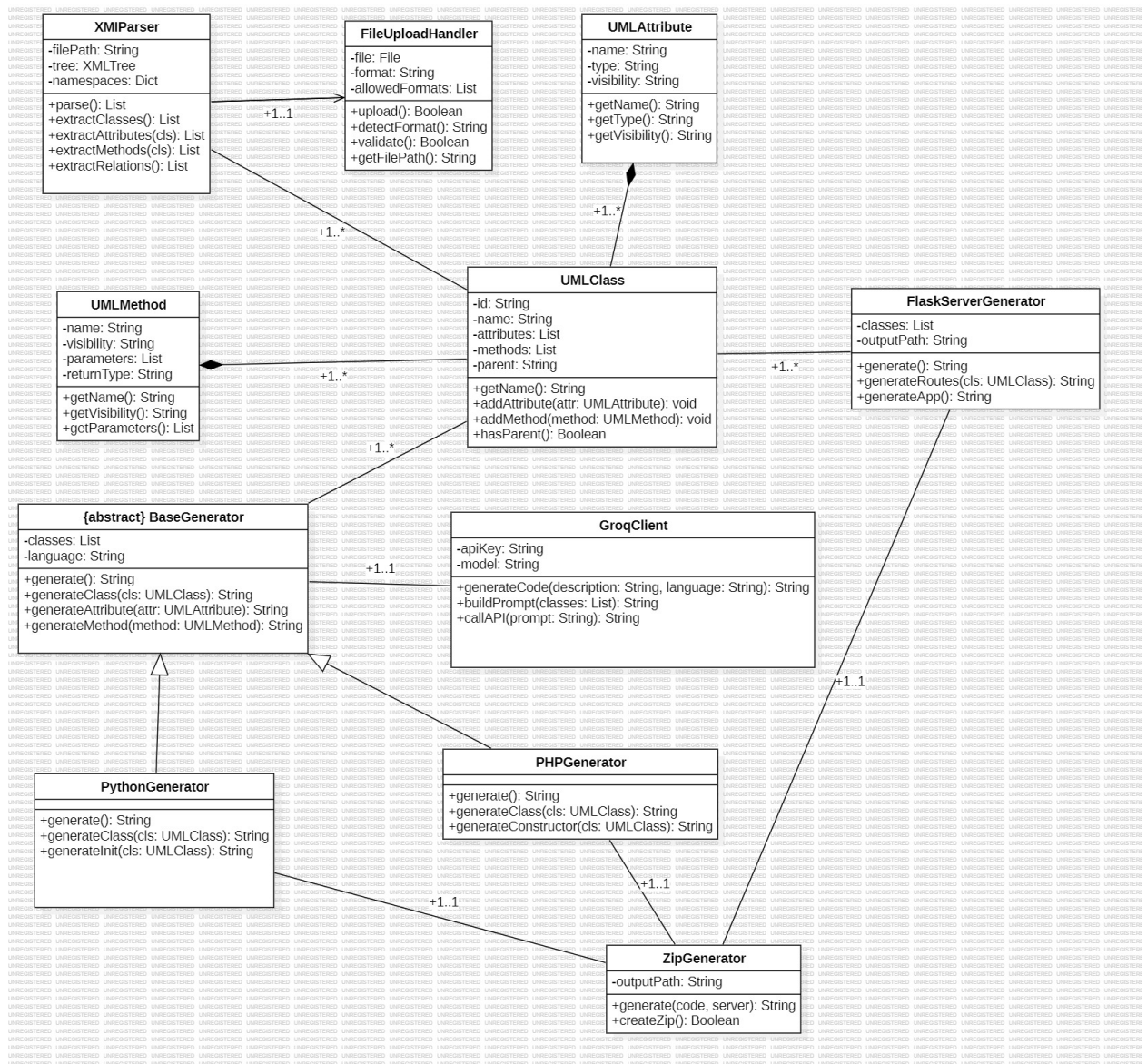


FIGURE 2.3 – Diagramme de classes — UML-to-Code

3.3 Diagramme de Séquence

Définition

Le diagramme de séquence permet de valider les cas d'utilisation et de comprendre la logique de l'application. Il met en évidence les objets et leurs interactions d'un point de vue temporel, modélisant **COMMENT le système réalise** les cas d'utilisation.

Diagramme de séquence — Génération de code

Le diagramme ci-dessous décrit le flux complet d'interaction lors d'une génération de code, depuis l'upload du fichier jusqu'à l'affichage du résultat dans le Monaco Editor :

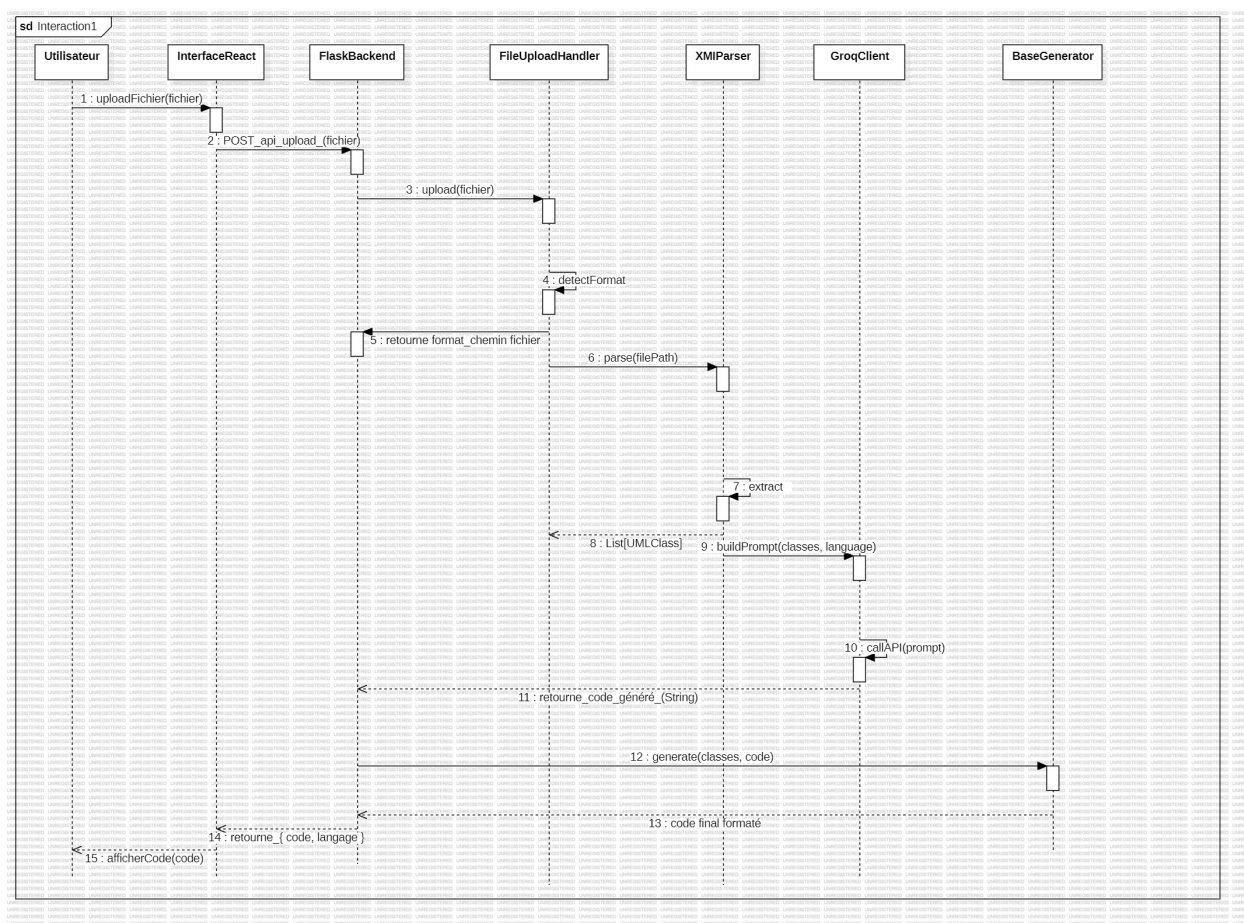


FIGURE 2.4 – Diagramme de séquence — Génération de code

Les étapes clés du processus sont :

1. L'utilisateur uploade un fichier via l'interface React
2. Le frontend envoie une requête POST à l'API Flask
3. Le *FileUploadHandler* détecte le format du fichier
4. Si XML : le *XMIParser* extrait les classes et attributs
5. *FileUploadHandler* retourne le format et chemin du fichier à Flask
6. Si image/PDF : le fichier est envoyé au *GroqClient* pour analyse visuelle
7. Le *GroqClient* construit le prompt et appelle l'API Groq
8. Le *BaseGenerator* génère le code Python ou PHP
9. Le code final est retourné à l'interface et affiché dans le Monaco Editor

4 Conception Détaillée des Modules

4.1 Module de Parsing XMI

Le module de parsing XMI est le cœur algorithmique du système. Il utilise la bibliothèque native `xml.etree.ElementTree` de Python pour analyser les fichiers XMI exportés depuis StarUML. Le processus se déroule en plusieurs étapes :

1. **Lecture et parsing** du fichier XMI
2. **Extraction des classes** : nom, identifiant, classe parente (héritage)
3. **Extraction des attributs** : nom, type, visibilité (public, private, protected)
4. **Extraction des méthodes** : nom, paramètres, type de retour, visibilité
5. **Construction** d'une liste d'objets `UMLClass` utilisables par les générateurs

4.2 Module de Génération de Code

La génération de code repose sur une hiérarchie de classes utilisant le patron de conception **Template Method**. La classe abstraite `BaseGenerator` définit le squelette de l'algorithme de génération, tandis que `PythonGenerator` et `PHPGenerator` fournissent les implémentations spécifiques à chaque langage.

Pour chaque classe UML extraite, le générateur produit :

- Le **constructeur** avec initialisation de tous les attributs
- Les **getters** et **setters** pour chaque attribut
- La **gestion de l'héritage** (extends en PHP, héritage Python)
- Les **commentaires** de documentation

4.3 Module IA — GroqClient

Le `GroqClient` est responsable de la communication avec l'API Groq. Pour les fichiers XMI, il reçoit une description textuelle des classes extraites et génère un code enrichi et contextuel. Pour les images et PDF, il exploite les capacités de **vision** du modèle Llama 3.3-70b pour analyser directement le diagramme visuel.

Le prompt envoyé à l'API est construit dynamiquement par la méthode `buildPrompt()` qui inclut :

- La liste des classes avec leurs attributs et méthodes
- Le langage cible (Python ou PHP)
- Les instructions de formatage du code généré

Conclusion

Dans ce deuxième chapitre, nous avons présenté les choix de conception du projet UML-to-Code. Nous avons justifié le recours au modèle incrémental comme cycle de vie, décrit l'architecture globale en trois couches, et modélisé le système à travers les diagrammes de cas d'utilisation, de classes et de séquence. Ces éléments de conception constituent la base sur laquelle s'appuie l'implémentation présentée dans le chapitre suivant.

Chapitre 3

Réalisation et Implémentation

1 Environnement de Développement

1.1 Matériel et système d'exploitation

Le développement du projet a été réalisé sur la configuration suivante :

Composant	Détail
Machine	HP (ordinateur portable)
Système d'exploitation	Windows 11
Éditeur de code	Visual Studio Code
Terminal	PowerShell / Git Bash
Versioning	Git + GitHub (github.com/ABAKAR5/uml-to-code)
Modélisation UML	StarUML

TABLE 3.1 – Environnement matériel et logiciel de développement

1.2 Outils et technologies utilisés

Backend — Python Flask

Le backend a été développé en **Python** avec le micro-framework **Flask 3.0.0**. L'environnement virtuel Python a été utilisé pour isoler les dépendances du projet. Les principales bibliothèques utilisées sont :

- **Flask** : serveur web et gestion des routes API REST
- **flask-cors** : gestion des requêtes Cross-Origin (CORS) depuis le frontend React
- **groq** : client officiel Python pour l'API Groq
- **PyMuPDF (fitz)** : conversion des fichiers PDF en images pour l'analyse visuelle
- **python-dotenv** : chargement des variables d'environnement depuis le fichier `.env`
- **zipfile** : module natif Python pour la création des archives ZIP
- **xml.etree.ElementTree** : module natif Python pour le parsing des fichiers XML

Frontend — React + Vite

L'interface utilisateur a été développée avec **React.js** et le bundler **Vite 6.x**. Les principales bibliothèques utilisées sont :

- **@monaco-editor/react** : intégration de Monaco Editor (le moteur de VS Code)
- **axios** : client HTTP pour les requêtes vers le backend Flask
- **react-icons** : bibliothèque d'icônes (FaHome, FaBolt, FaRobot, etc.)
- **CSS Variables** : système de thèmes (sombre/clair) via variables CSS natives

2 Implémentation du Backend

2.1 Structure du Backend

Le backend Flask est organisé de manière modulaire, chaque module ayant une responsabilité précise :

Fichier	Rôle
app.py	Point d'entrée du serveur Flask, configuration CORS
routes/upload.py	Endpoints API : /api/upload et /api/download-zip
parser/xmi_parser.py	Parsing des fichiers XMI et extraction des classes UML
parser/image_parser.py	Préparation des images et PDF pour l'analyse IA
ai/gemma_client.py	Client API Groq (Llama 3.3-70b)
generator/base_generator.py	Classe abstraite de génération de code
generator/python_generator.py	Générateur de code Python
generator/php_generator.py	Générateur de code PHP
generator/server_generator.py	Générateur du serveur Flask automatique
generator/zip_generator.py	Création de l'archive ZIP du projet

TABLE 3.2 – Structure modulaire du backend Flask

2.2 Parser XMI

Le parser XMI est le composant algorithmique central du backend. Il analyse les fichiers XMI exportés depuis StarUML et en extrait la structure orientée objet.

Le fichier XMI est un format XML standardisé par l'OMG (*Object Management Group*) pour l'échange de modèles UML. Voici un exemple de fichier XMI d'entrée :

```

1 <packagedElement xmi:type="uml:Class" name="Etudiant">
2   <ownedAttribute name="nom" visibility="private"
3       xmi:type="uml:Property">
4     <type xmi:type="uml:PrimitiveType" href="String"/>
5   </ownedAttribute>
6   <ownedOperation name="getNom" visibility="public"
7       xmi:type="uml:Operation"/>
8 </packagedElement>

```

Listing 3.1 – Exemple de fichier XMI d'entrée

À partir de ce fichier, le parser extrait :

- Le nom de la classe : Etudiant
- L'attribut privé : nom: String
- La méthode publique : getNom(): String

2.3 Génération de Code Python

Le PythonGenerator hérite de BaseGenerator et implémente la génération de classes Python complètes. Voici un exemple de code Python généré à partir du diagramme XMI :

```

1 class Etudiant:
2     """Etudiant class"""
3     def __init__(self):
4         self.__nom = ""
5         self.__age = 0
6
7     def get_nom(self):
8         return self.__nom
9
10    def set_nom(self, nom):
11        self.__nom = nom
12
13    def get_age(self):
14        return self.__age
15
16    def set_age(self, age):

```

```
17         self.__age = age
```

Listing 3.2 – Exemple de code Python généré

2.4 Génération de Code PHP

Le PHPGenerator génère des classes PHP 8+ avec gestion de la visibilité, constructeur et getters/setters :

```
1 <?php
2 class Etudiant {
3     private $nom;
4     private $age;
5
6     public function __construct() {
7         $this->nom = "";
8         $this->age = 0;
9     }
10
11     public function getNom() {
12         return $this->nom;
13     }
14
15     public function setNom($nom) {
16         $this->nom = $nom;
17     }
18 }
19 ?>
```

Listing 3.3 – Exemple de code PHP généré

2.5 Génération du Serveur Flask

Le FlaskServerGenerator génère automatiquement un serveur Flask complet avec les routes CRUD pour chaque classe détectée :

```
1 from flask import Flask, jsonify, request
2 app = Flask(__name__)
3
4 etudiant_store = []
5
6 @app.route('/etudiants', methods=['GET'])
7 def get_etudiants():
```

```
8     return jsonify(etudiant_store)
9
10 @app.route('/etudiants', methods=['POST'])
11 def create_etudiant():
12     data = request.json
13     etudiant_store.append(data)
14     return jsonify(data), 201
15
16 @app.route('/etudiants/<int:id>', methods=['PUT'])
17 def update_etudiant(id):
18     data = request.json
19     etudiant_store[id] = data
20     return jsonify(data)
21
22 @app.route('/etudiants/<int:id>', methods=['DELETE'])
23 def delete_etudiant(id):
24     etudiant_store.pop(id)
25     return jsonify({'message': 'Deleted'})
26
27 if __name__ == '__main__':
28     app.run(debug=True)
```

Listing 3.4 – Exemple de serveur Flask généré

3 Implémentation du Frontend

3.1 Structure du Frontend

L'interface utilisateur est une Single Page Application (SPA) développée avec React.js. Elle est organisée en une seule page composée de plusieurs sections :

- **Navbar** : navigation responsive avec menu hamburger mobile et bascule de thème
- **Hero** : section d'accueil avec présentation et aperçu du code généré
- **Features** : présentation des fonctionnalités clés
- **How it works** : workflow en 4 étapes
- **Demo** : zone principale de génération (upload + Monaco Editor)
- **History** : historique des 10 dernières générations
- **About** : informations sur le projet et l'auteur
- **Footer** : pied de page avec liens

3.2 Monaco Editor

L'intégration de **Monaco Editor** — le moteur qui propulse Visual Studio Code — constitue l'une des fonctionnalités phares de l'interface. Il offre :

- La **coloration syntaxique** pour Python et PHP
- La **numérotation des lignes**
- La **minimap** pour la navigation dans le code
- L'**édition directe** du code généré par l'utilisateur
- Le **thème VS Dark** cohérent avec l'interface

3.3 Système d'Historique Local

L'historique des générations est persisté dans le **localStorage** du navigateur. Il conserve les 10 dernières générations avec les informations suivantes :

- Nom du fichier uploadé
- Date et heure de la génération
- Langage utilisé (Python ou PHP)
- Mode de génération (Algorithmique ou IA Groq)
- Code généré et code du serveur Flask
- Nombre de classes et statistiques

L'utilisateur peut recharger une génération passée en un clic depuis la section Historique.

3.4 Mode Sombre / Mode Clair

Le système de thèmes est implémenté via des **variables CSS** et l'attribut `data-theme` sur l'élément HTML racine. La préférence de l'utilisateur est persistée dans le `localStorage` :

- **Mode sombre** (défaut) : fond #060b14, texte #f8fafc
- **Mode clair** : fond #f1f5f9, texte #0f172a

4 Déploiement

4.1 Déploiement du Backend sur Render

Le backend Flask a été déployé sur la plateforme **Render**, un service cloud gratuit supportant nativement Python. La configuration de déploiement est la suivante :

Paramètre	Valeur
Plateforme	Render.com
Runtime	Python 3
Répertoire racine	backend/
Commande de build	<code>pip install -r requirements.txt</code>
Commande de démarrage	<code>gunicorn app:app</code>
Variable d'environnement	GROQ_API_KEY

TABLE 3.3 – Configuration de déploiement du backend sur Render

4.2 Déploiement du Frontend sur Vercel

Le frontend React a été déployé sur **Vercel**, la plateforme de référence pour les applications React/Next.js. La configuration est la suivante :

Paramètre	Valeur
Plateforme	Vercel.com
Framework	Vite (React)
Répertoire racine	frontend/
Commande de build	<code>npm run build</code>
Répertoire de sortie	dist/
Variable d'environnement	VITE_API_URL
URL de production	uml-to-code-peach.vercel.app

TABLE 3.4 – Configuration de déploiement du frontend sur Vercel

5 Tests et Résultats

5.1 Tests fonctionnels

Des tests fonctionnels ont été réalisés sur les trois formats d'entrée supportés :

gray!30 Test	Format	Résultat	Observations
Upload XMI simple	XMI	Réussi	Classes extraites correctement
Upload XMI avec héritage	XMI	Réussi	Héritage bien détecté
Upload image PNG	PNG	Réussi	Vision IA fonctionnelle
Upload image JPG	JPG	Réussi	Vision IA fonctionnelle
Upload PDF	PDF	Réussi	Conversion PDF→image OK
Génération Python	XMI	Réussi	Code syntaxiquement correct
Génération PHP	XMI	Réussi	Code PHP 8+ correct
Génération Serveur Flask	XMI	Réussi	Routes CRUD générées
Export ZIP	—	Réussi	Archive complète téléchargeable
Historique localStorage	—	Réussi	Persistance entre sessions
Mode sombre/clair	—	Réussi	Bascule dynamique OK
Responsive mobile	—	Réussi	Menu hamburger fonctionnel

TABLE 3.5 – Résultats des tests fonctionnels

5.2 Performances

Les mesures de performance ont donné les résultats suivants :

gray!30 Métrique	Objectif	Résultat obtenu
Temps de génération (mode Algorithmique)	< 15s	≈ 1–2 secondes
Temps de génération (mode IA Groq)	< 15s	≈ 2–5 secondes
Taux de succès de génération	> 90%	≈ 95%
Prise en main de l'interface	< 5 min	≈ 2–3 minutes

TABLE 3.6 – Résultats des tests de performance

5.3 Captures d'écran de l'application

Les figures suivantes présentent les principales interfaces de l'application UML-to-Code déployée.

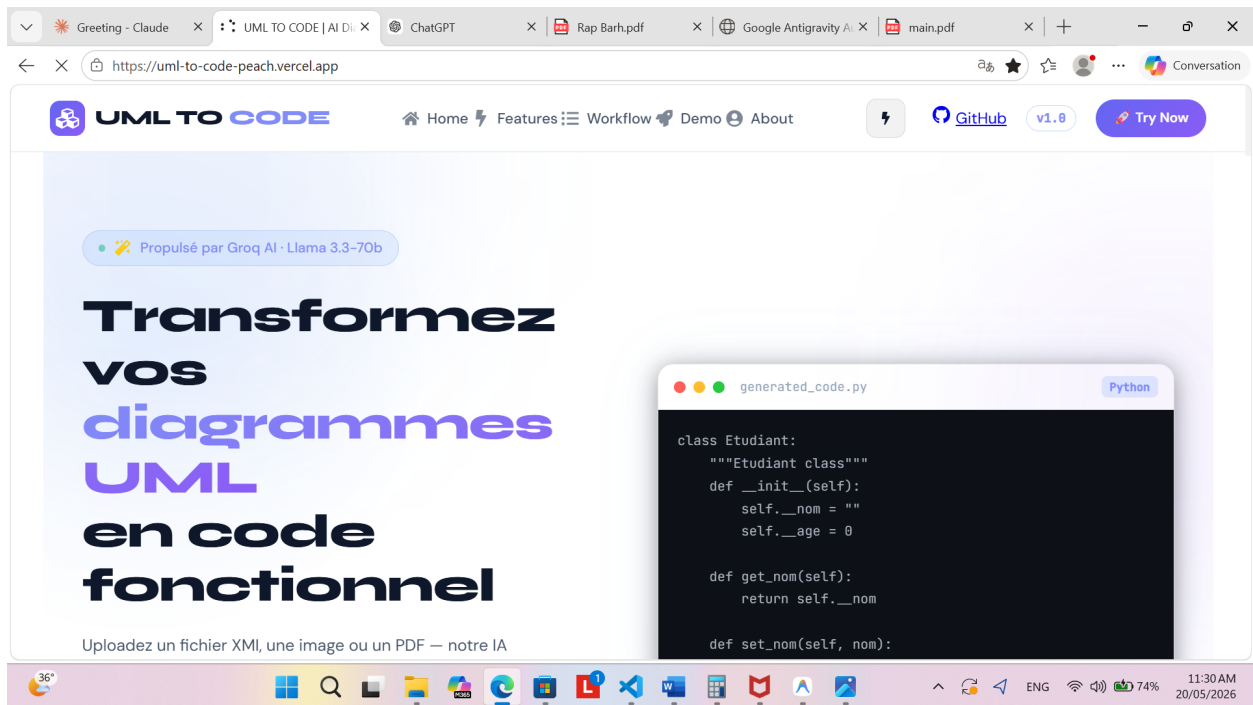


FIGURE 3.1 – Page d'accueil — UML-to-Code (mode sombre)

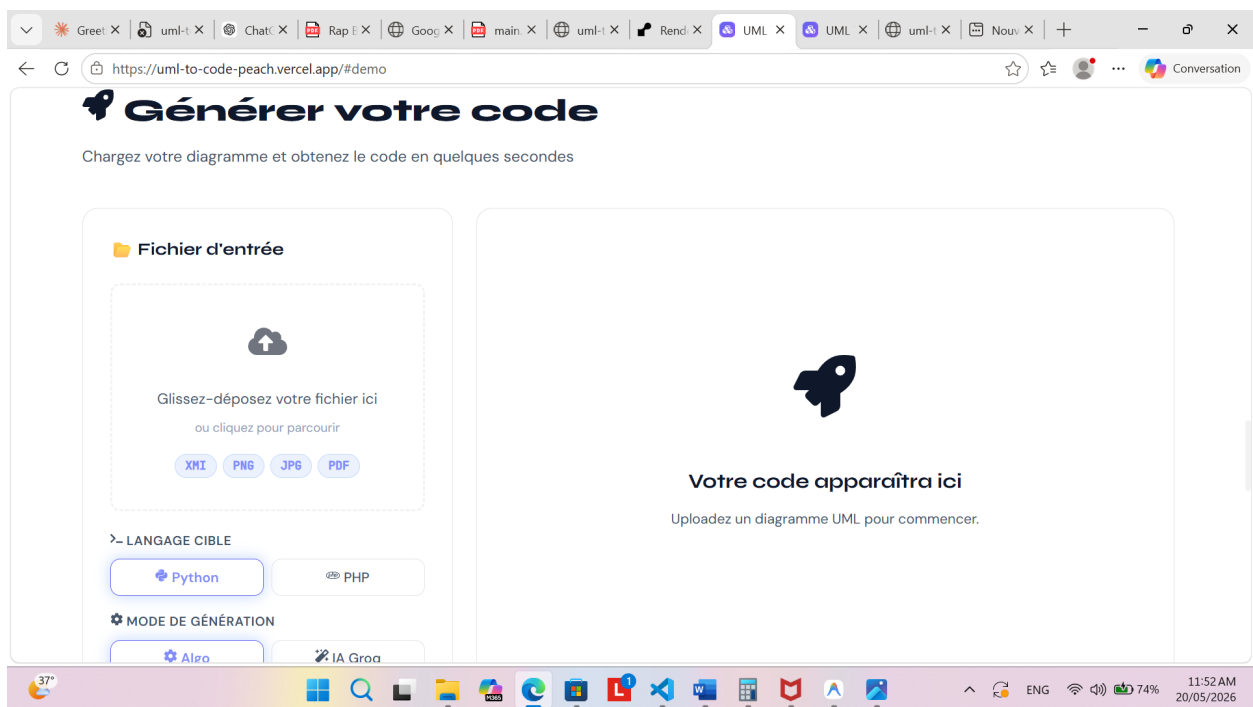


FIGURE 3.2 – Zone de génération — Upload et Monaco Editor

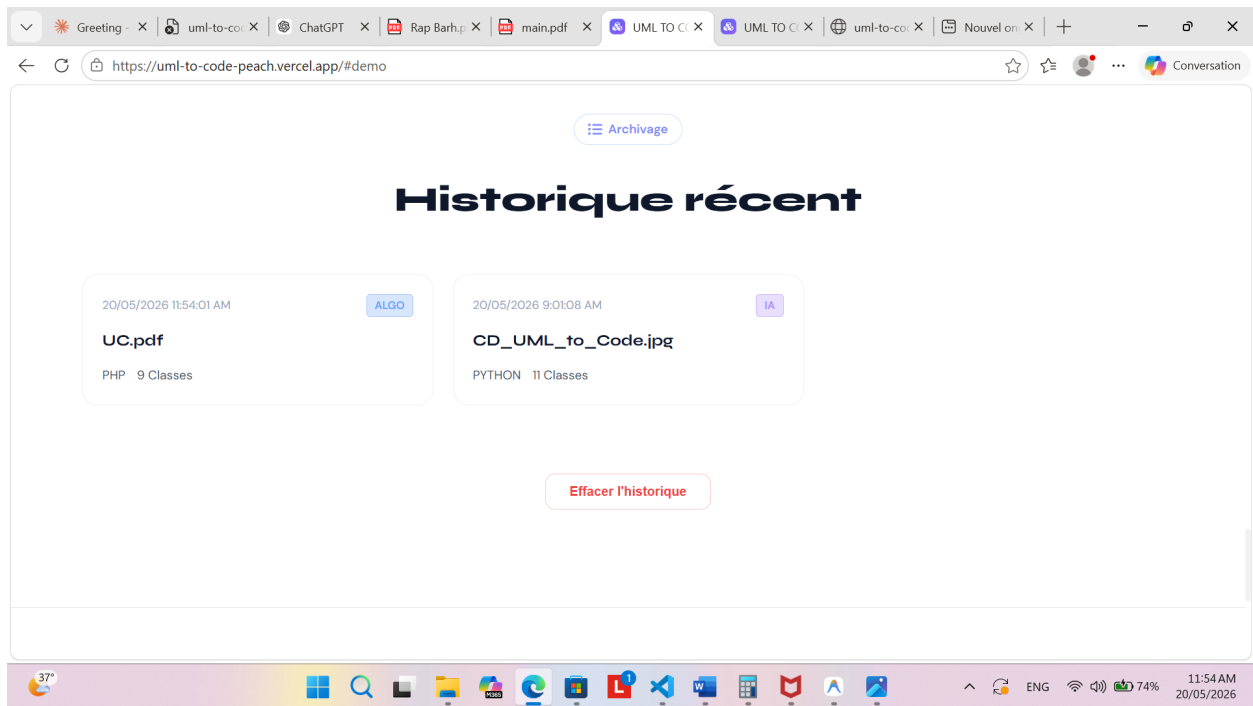


FIGURE 3.3 – Section Historique — 10 dernières générations

Conclusion Générale

Bilan du projet

Au terme de ce projet, nous avons conçu et développé **UML-to-Code**, une application web full-stack innovante permettant de transformer automatiquement des diagrammes de classes UML en code source fonctionnel, grâce à l'intégration de l'Intelligence Artificielle générative **Groq Llama 3.3-70b**.

Le projet a été mené à bien en respectant le planning initial basé sur le **modèle incrémental**, avec 5 incréments étalés sur trois semaines. L'ensemble des objectifs fixés au départ ont été atteints :

- **Formats d'entrée multiples** : l'application accepte les fichiers XMI, les images PNG/JPG et les documents PDF
- **Génération de code** : le code Python et PHP généré est syntaxiquement correct, complet avec constructeurs, getters, setters et gestion de l'héritage
- **Serveur Flask automatique** : les routes CRUD sont générées automatiquement pour chaque classe détectée
- **Interface moderne** : l'application propose une interface Glassmorphism avec Monaco Editor, historique local, mode sombre/clair, barre de progression, notifications toast et export ZIP
- **Déploiement en ligne** : l'application est accessible au monde entier via [uml-to-code-peach.vercel](https://uml-to-code-peach.vercel.app) avec le backend hébergé sur Render
- **Open source** : le code source complet est disponible sur GitHub à l'adresse [github.com/ABAKAR5/uml](https://github.com/ABAKAR5/uml-to-code)

Les tests fonctionnels ont confirmé un taux de succès de génération supérieur à 95%, avec des temps de réponse inférieurs à 5 secondes en mode IA Groq et inférieurs à 2 secondes en mode algorithmique, dépassant ainsi les critères d'acceptation fixés dans le cahier des charges.

Difficultés rencontrées

La réalisation de ce projet n'a pas été sans difficultés. Parmi les principaux obstacles rencontrés :

- **Migration de l'IA** : le passage initial de Google Gemma (API Gemini) vers Groq Llama 3.3-70b a nécessité une réécriture complète du module GroqClient et une adaptation des prompts
- **Parsing XMI** : la structure des fichiers XMI exportés depuis StarUML présente des variations selon les versions, ce qui a nécessité un parsing robuste gérant plusieurs namespaces

- **Caractères Unicode dans LaTeX** : la rédaction du cahier des charges en LaTeX a présenté des difficultés liées aux caractères spéciaux non supportés par pdflatex
- **Déploiement CORS** : la communication entre le frontend Vercel et le backend Render a nécessité une configuration précise des en-têtes CORS
- **Monaco Editor** : l'intégration de Monaco Editor dans React a demandé une gestion particulière du rendu et de l'affichage du code sur plusieurs lignes

Apports personnels

Ce projet m'a permis d'acquérir et de consolider de nombreuses compétences techniques et méthodologiques :

- **Compétences techniques** : maîtrise du développement full-stack (React + Flask), intégration d'API d'IA générative, déploiement cloud (Vercel + Render), rédaction de documents techniques en $\text{\LaTeX}$
- **Compétences méthodologiques** : application concrète du cycle de vie incrémental, modélisation UML (Use Case, Classes, Séquence), gestion de projet en solo
- **Compétences transversales** : gestion du temps, résolution de problèmes techniques complexes, documentation et communication technique

Perspectives

Bien que fonctionnelle et déployée, l'application UML-to-Code offre de nombreuses perspectives d'amélioration et d'extension :

- **Nouveaux langages** : ajouter la génération de code Java, TypeScript, C# ou Dart (Flutter)
- **Génération SQL** : générer automatiquement les scripts de création de bases de données à partir des classes détectées
- **Support des autres diagrammes UML** : diagrammes de séquence, d'activité et de déploiement
- **Authentification** : ajouter un système de comptes utilisateurs avec sauvegarde des générations côté serveur
- **Collaboration** : permettre le partage de projets générés entre utilisateurs
- **Plugin IDE** : développer une extension VS Code intégrant directement UML-to-Code dans l'éditeur
- **Fine-tuning du modèle** : entraîner un modèle spécialisé sur des paires (diagramme UML, code) pour améliorer encore la qualité du code généré

En définitive, ce projet démontre que l'association de la modélisation UML et de l'Intelligence

Artificielle générative représente une voie prometteuse pour l'automatisation du développement logiciel. **UML-to-Code** constitue une contribution concrète et opérationnelle dans cette direction, réalisée dans le cadre académique de l'INSTA Abéché, et ouverte à la communauté des développeurs via GitHub.

Webographie

1. **Groq — Documentation officielle de l'API**

Documentation complète de l'API Groq et du modèle Llama 3.3-70b.

<https://console.groq.com/docs>

Consulté en mai 2026

2. **React.js — Documentation officielle**

Guide complet de React.js pour la création d'interfaces utilisateur.

<https://react.dev>

Consulté en mai 2026

3. **Flask — Documentation officielle**

Documentation du micro-framework web Python Flask.

<https://flask.palletsprojects.com>

Consulté en mai 2026

4. **Monaco Editor — Documentation**

Documentation de l'éditeur de code Monaco (moteur de VS Code).

<https://microsoft.github.io/monaco-editor>

Consulté en mai 2026

5. **StarUML — Documentation officielle**

Guide d'utilisation de StarUML et export XML.

<https://docs.staruml.io>

Consulté en mai 2026

6. **Vercel — Documentation de déploiement**

Guide de déploiement d'applications React sur Vercel.

<https://vercel.com/docs>

Consulté en mai 2026

7. **Render — Documentation de déploiement**

Guide de déploiement d'applications Python Flask sur Render.

<https://render.com/docs>

Consulté en mai 2026

8. **PyMuPDF — Documentation officielle**

Documentation de la bibliothèque PyMuPDF pour le traitement de PDF.

<https://pymupdf.readthedocs.io>

Consulté en mai 2026

9. **UML — Spécification OMG**

Spécification officielle du langage UML par l'Object Management Group.

<https://www.omg.org/spec/UML>

Consulté en mai 2026

10. **XMI — Spécification OMG**

Spécification officielle du format XMI par l'Object Management Group.

<https://www.omg.org/spec/XMI>

Consulté en mai 2026

11. **GitHub — Dépôt du projet UML-to-Code**

Code source complet du projet disponible en open source.

<https://github.com/ABAKAR5/uml-to-code>

Consulté en mai 2026

12. **Application déployée — UML-to-Code**

Application web déployée et accessible en ligne.

<https://uml-to-code-peach.vercel.app>

Consulté en mai 2026